

APPLICAZIONE MATCHUP

CORSO DI INGEGNERIA DEL SOFTWARE A.A. 2025 / 2026

DOCUMENTO DI PROGETTO ITERAZIONE 1

SALVATORE CAMPIONE

MARIO CIPRIANO

Sommario

1 REQUISITI	3
2. ANALISI	3
2.1 MODELLO DI DOMINIO	3
2.2 DIAGRAMMA DI SEQUENZA DI SISTEMA UC3	4
2.3 CONTRATTI DELLE OPERAZIONI DI SISTEMA UC3	5
2.3.1 EFFETTUA RICERCA STRUTTURA	5
2.3.2 PRENOTA CAMPO	5
2.3.3 CONFERMA PRENOTAZIONE	6
3 PROGETTAZIONE	6
3.1 DIAGRAMMA DI SEQUENZA UC3	7
3.1.1 EFFETTUA RICERCA STRUTTURA	7
3.1.2 PRENOTA CAMPO	7
3.1.3 CONTROLLA DISPONIBILITA' CAMPO	8
3.1.4 CONFERMA PRENOTAZIONE	8
3.2 DIAGRAMMA DELLE CLASSI DI PROGETTO	9
4 FASE DI TESTING	9
4.1 INTRODUZIONE E STRATEGIA DI TESTING	9
4.2 SCENARIO PRENOTA CAMPO	10

1 REQUISITI

In questa prima iterazione implementiamo lo scenario di successo del caso d'uso UC3, concentrandoci su tali obiettivi:

- Prenotare un campo in una determinata struttura
- Calcolo del costo della prenotazione sulla base alle regole di dominio

Per semplicità, in questa iterazione verrà trascurato la registrazione della struttura e dell'utente e l'eventuale autenticazione all'interno del sistema: infatti verrà impostato un caso d'uso di avviamento che consiste nell'istanziamento di alcune strutture di test contenente una serie di campi che si proveranno a prenotare e si implementerà un login successivamente.

2. ANALISI

In questa sezione verranno riportati gli elaborati UML della fase di analisi. Verranno mostrati il modello di dominio, i diagrammi di sequenza di sistema e i contratti delle operazioni di sistema individuati.

2.1 MODELLO DI DOMINIO

In questa prima iterazione, dall'UC3, sono state individuate le seguenti classi progettuali:

- MatchUp: si tratta del SuD;
- Giocatore: persona che intende effettuare una prenotazione;
- Campo: oggetto che viene prenotato dal giocatore in una certa data e fascia oraria ,a patto che sia disponibile .
- Struttura: impresa che possiede un certo numero di campi;
- Prenotazione: noleggio del campo effettuato dal giocatore

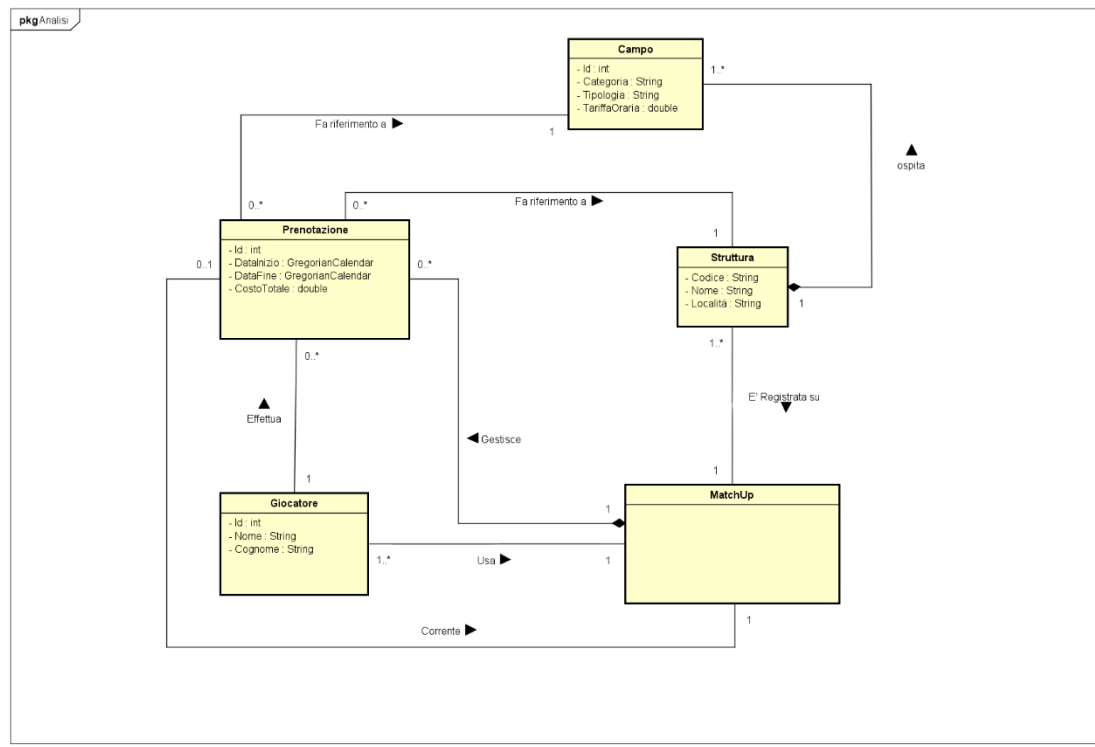


Figura 1 Modello di dominio

2.2 DIAGRAMMA DI SEQUENZA DI SISTEMA UC3

Di seguito viene riportato il diagramma di sequenza di sistema del caso d'uso UC3

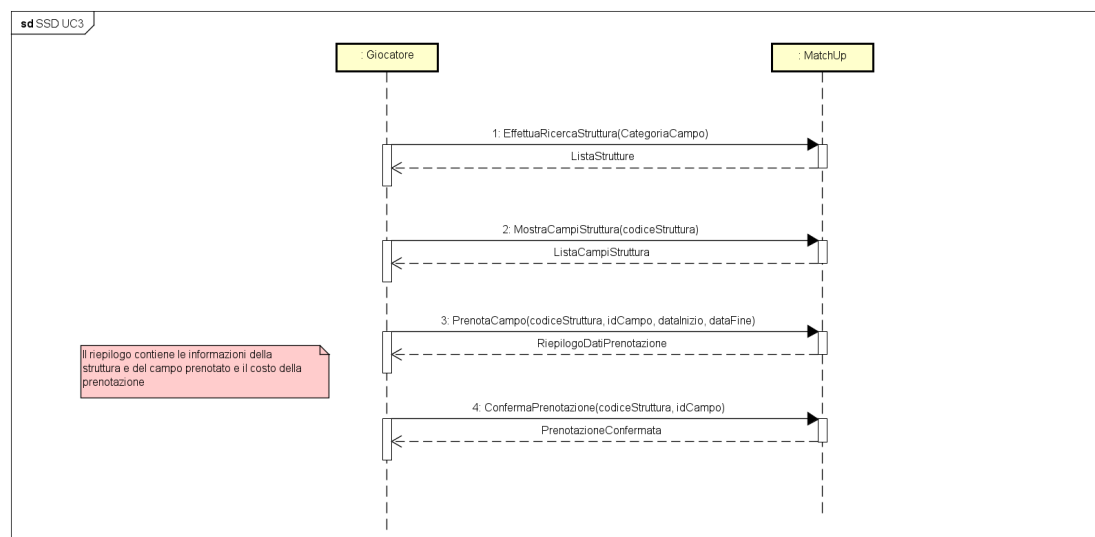


Figura 2 Diagramma di sequenza di sistema UC3

2.3 CONTRATTI DELLE OPERAZIONI DI SISTEMA UC3

2.3.1 EFFETTUA RICERCA STRUTTURA

Questa operazione permette, a partire dalla categoria del campo richiesto, di trovare e ritornare una lista di strutture che possiedono almeno un campo di quella categoria.

Operazione	EffettuaRicercaStruttura(categoriaCampo:String)
Riferimenti	Caso d'uso UC3 (EffettuaPrenotazione)
Pre-condizioni	- Sono state istanziate una o più strutture di test, ciascuna con i relativi campi . - Il parametro Categoria è un valore valido/non nullo.
Post-condizioni	- E' stata creata una lista di istanze di strutture idonee alla richiesta del giocatore

Tabella 1 Contratto operazione EffettuaRicercaStruttura

2.3.2 PRENOTA CAMPO

Questa operazione permette al giocatore di riservare il campo per l'ora e data richiesta.

Operazione	PrenotaCampo(CodiceStruttura:String,IdCampo:int, DataInizio:GregorianCalendar,DataFine:GregorianCalendar)
Riferimenti	Caso d'uso UC3 (EffettuaPrenotazione)
Pre-condizioni	- Il giocatore ha scelto il campo che intende prenotare per la data e l'ora richiesta
Post-condizion	- E' stata creata un'istanza di prenotazione p - Gli attributi di p sono stati inizializzati - p è stata associata a Matchup tramite l'associazione corrente

Tabella 2 Contratto operazione PrenotaCampo

2.3.3 CONFERMA PRENOTAZIONE

Questa operazione permette di confermare i dati della prenotazione che verrà poi registrata nel sistema

Operazione	ConfermaPrenotazione(CodiceStruttura:String,IdCampo:int)
Riferimenti	Caso d'uso UC3 (EffettuaPrenotazione)
Pre-condizioni	- E' in corso una prenotazione p
Post-condizioni	- La prenotazione è stata associata a Matchup tramite l'associazione "Gestisce". - p è stata associata a campo tramite l'associazione "Fa riferimento a" - p è stata associata alla struttura s tramite l'associazione "Fa riferimento a" - p è stata associata al giocatore g1 tramite l'associazione "Effettua"

Tabella 3 Contratto operazione ConfermaPrenotazione

3 PROGETTAZIONE

Nella fase di progettazione verranno inseriti due tipologie di diagrammi :

- Diagrammi di sequenza: Questi modellano gli aspetti relativi della progettazione
- Diagrammi di classe di progetto: Questi invece definisco le classi software che andremo ad implementare.

3.1 DIAGRAMMA DI SEQUENZA UC3

3.1.1 EFFETTUA RICERCA STRUTTURA

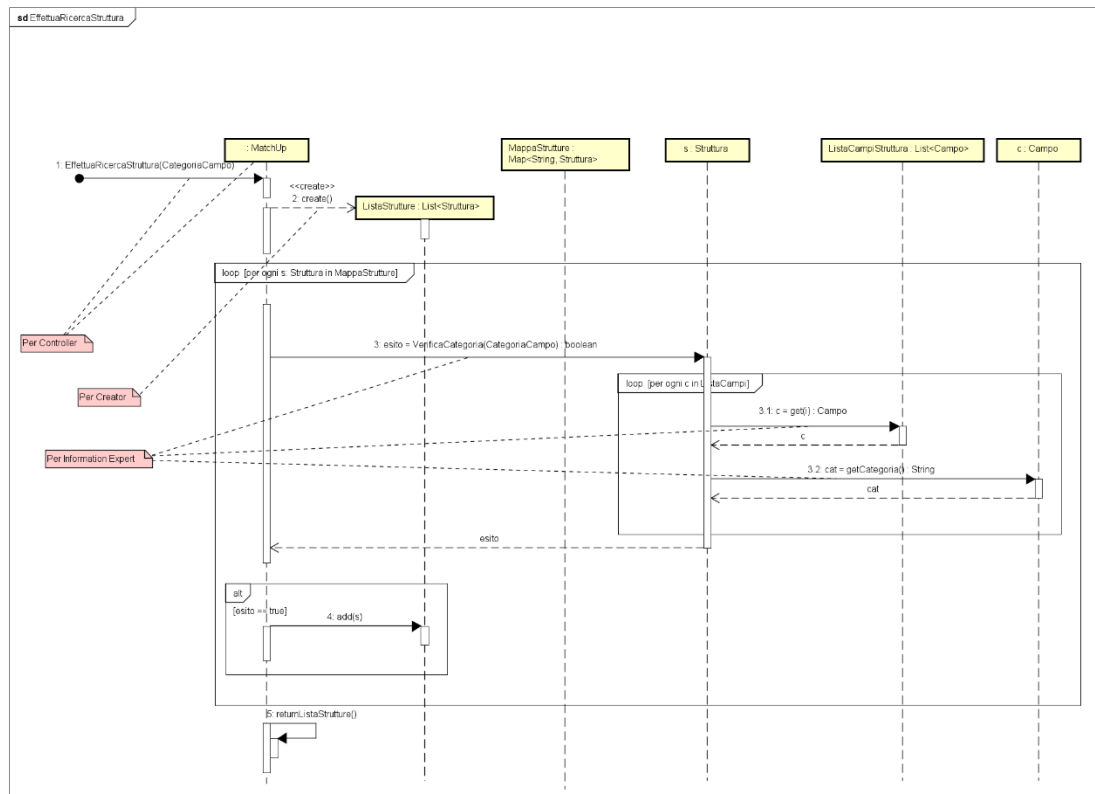


Figura 3 : Diagramma di sequenza Effettua ricerca struttura

3.1.2 PRENOTA CAMPO

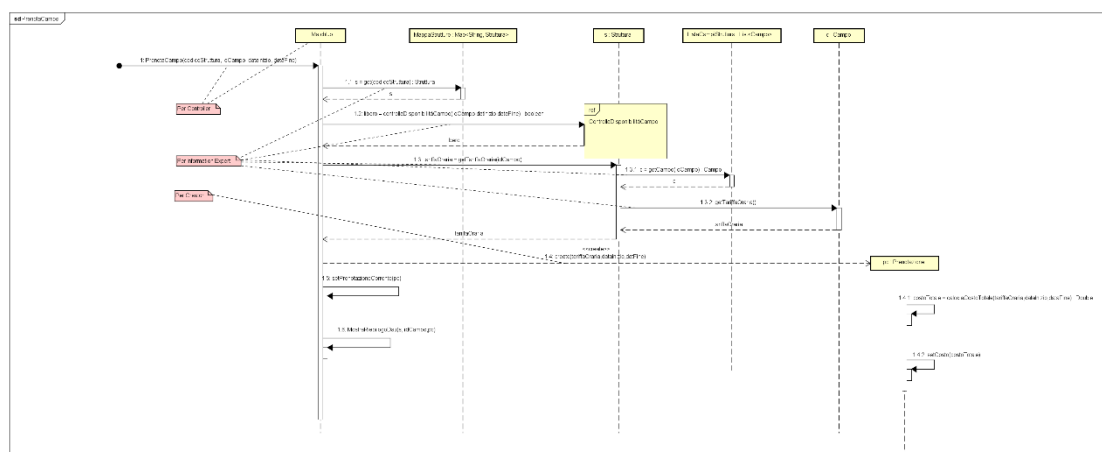


Figura 4: Diagramma di sequenza PrenotaCampo

Qui in prenota campo come possiamo vedere come abbiamo utilizzato un frame ref “controlla disponibilità campo”. Esso serve principalmente a mantenere il diagramma principale leggibile, evitando di espandere il loop di verifica sulle prenotazioni esistenti e isolare una responsabilità autonoma di Campo (Information Expert, poiché possiede listaPrenotazioniCampo), così documentata una sola volta e riferibile ovunque serva.

3.1.3 CONTROLLA DISPONIBILITA'CAMPO

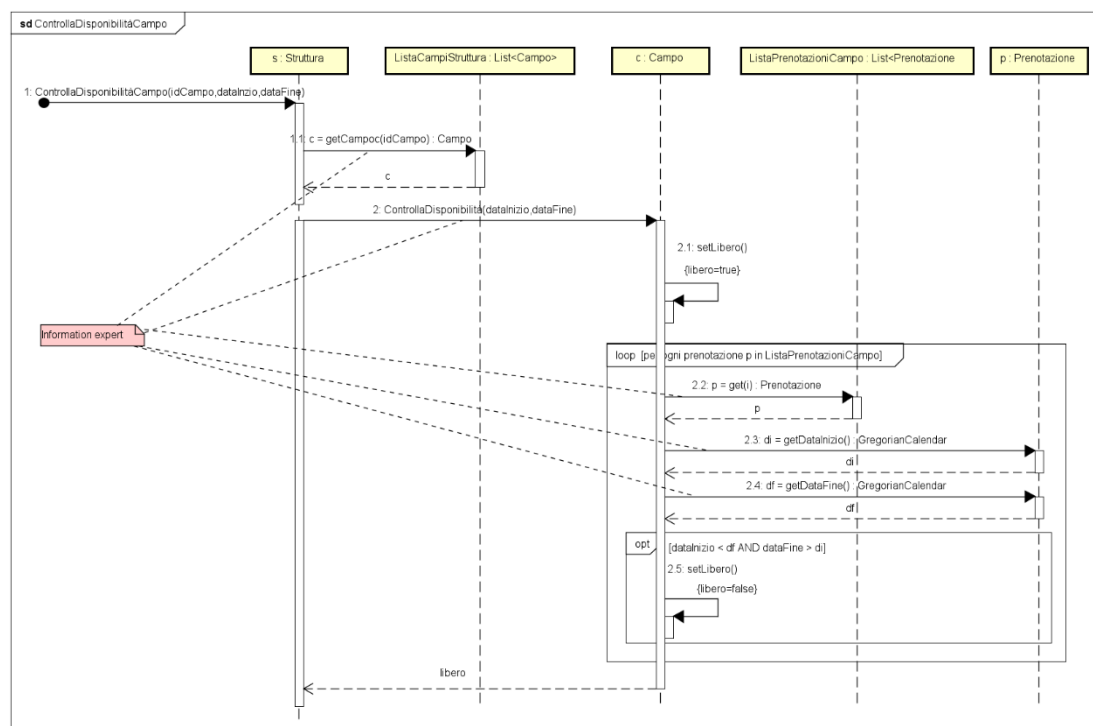


Figura 5: Diagramma di sequenza ControllaDisponibilitaCampo

3.1.4 CONFERMA PRENOTAZIONE

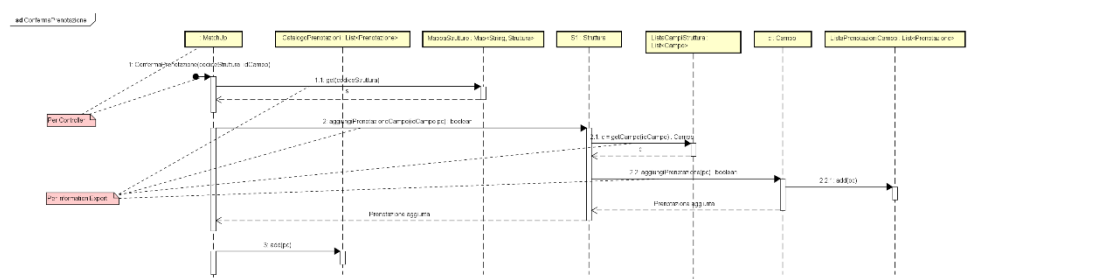


Figura 6: Diagramma di sequenza ConfermaPrenotazione

3.2 DIAGRAMMA DELLE CLASSI DI PROGETTO

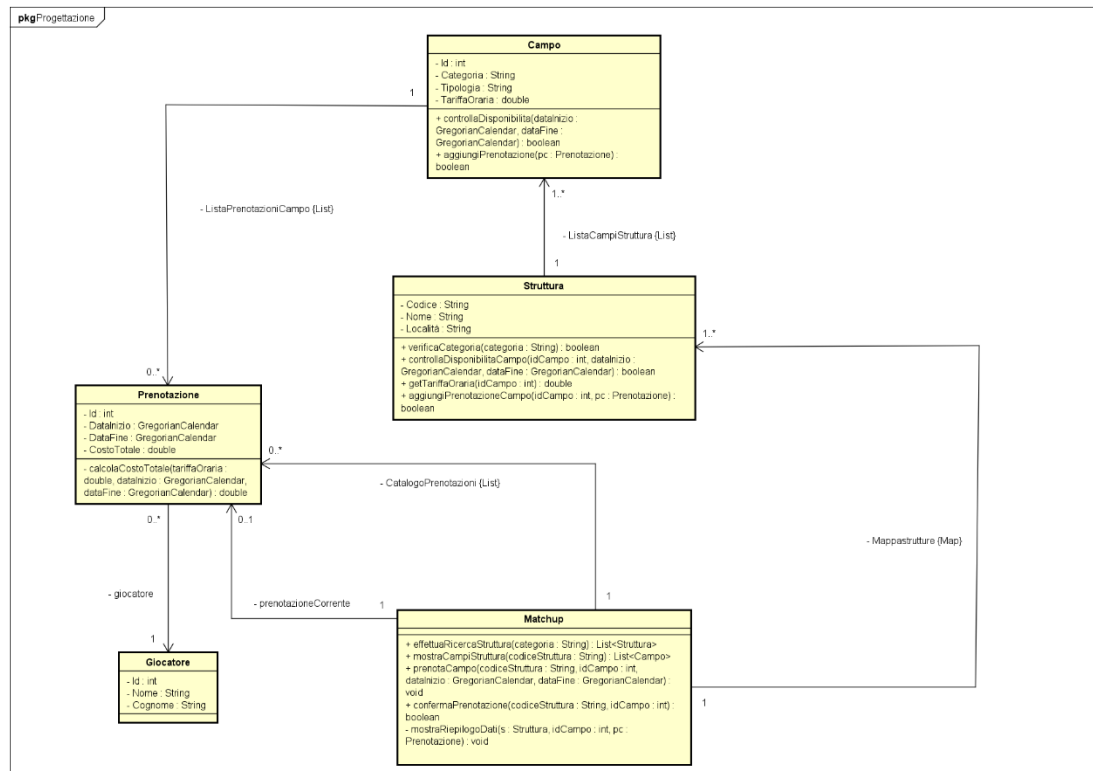


Figura 7 Diagramma delle classi di progetto

4 FASE DI TESTING

4.1 INTRODUZIONE E STRATEGIA DI TESTING

In questa prima iterazione, le funzionalità dell'applicazione sono state focalizzate sullo scenario principale del caso d'uso UC3 – Prenota Campo. Trattandosi di un'architettura iniziale, l'approccio alla definizione dei test case si è basato sull'analisi dei metodi caratterizzanti delle classi di dominio coinvolte: Campo, Struttura, Prenotazione e la classe controller del sistema MatchUp.

I test funzionali sono stati implementati tramite il framework JUnit. La strategia adottata prevede un approccio di Testing Bottom-Up (o a metodi annidati):

anziché testare unicamente le macro-funzionalità esposte dal controller, sono stati testati in primis i metodi interni e più specifici delle classi di dominio, per poi risalire verso i metodi più generali.

4.2 SCENARIO PRENOTA CAMPO

Lo scenario di test copre il ciclo di vita della prenotazione, articolato nei tre passaggi fondamentali previsti dal requisito:

Passaggio 1: Ricerca e Visualizzazione Strutture

Avviene tramite il metodo `effettuaRicercaStruttura(categoria)` testato nella classe `MatchUpTest`. Tale metodo interroga ogni struttura registrata invocando il metodo interno `verificaCategoria(categoria)` (testato in `StrutturaTest`).

- *Test su `verificaCategoria (StrutturaTest)`*: Verifica se la struttura possiede almeno un campo della categoria cercata. Esiti: `true` se presente, `false` altrimenti.
- *Test su `effettuaRicercaStruttura (MatchUpTest)`*: Verifica il comportamento della lista restituita. Esiti: lista popolata con le strutture idonee se esistono corrispondenze; lista vuota (`isEmpty() == true`) nel caso in cui nessuna struttura contenga campi per la categoria specificata.

Passaggio 2: Generazione e Validazione della Prenotazione (`prenotaCampo`)

Il metodo `prenotaCampo()` rappresenta il cuore del processo e sintetizza differenti punti di controllo affidati a classi distinte lungo la catena di chiamata:

- *a) Codice Struttura non esistente*: La verifica è a carico di `MatchUpTest`. Se il codice non trova riscontro in `mappaStrutture`, il controller interrompe l'esecuzione e restituisce `false`.
- *b) Codice Campo non esistente*: Se la struttura esiste, la responsabilità passa al metodo `controllaDisponibilitaCampo()` testato in `StrutturaTest`. Se l'ID campo non è presente tra quelli della struttura, il metodo ritorna `false`.
- *c) Fascia oraria occupata*: Se la struttura e il campo esistono, la verifica passa al metodo `controllaDisponibilita()` dell'entità `Campo` (testato in `CampoTest`). Se la fascia oraria richiesta si sovrappone a prenotazioni già esistenti, il metodo restituisce `false`.

- *d) Parametri validi e orario libero (Happy Path):* Se tutti i controlli precedenti hanno esito positivo, controllaDisponibilita() restituisce true, e MatchUp istanzia temporaneamente la prenotazione assegnandola all'attributo prenotazioneCorrente.

Passaggio 3: Verifiche sul Calcolo del Prezzo e Test di Sistema

Il calcolo del prezzo totale viene eseguito mediante il metodo calcolaCostoTotale() della classe Prenotazione.

- *Nota sulla visibilità:* Poiché calcolaCostoTotale() è un metodo private incapsulato all'interno dell'oggetto Prenotazione, non è direttamente invocabile tramite Unit Test standard di JUnit al di fuori della classe stessa.
- *Strategia applicata:* La corretta determinazione del costo (durata × tariffa oraria) è stata pertanto verificata tramite un Test di Sistema a livello di integrazione, invocando il flusso pubblico di prenotazione e confrontando il prezzo generato nell'istanza di prenotazioneCorrente con il valore teorico atteso dalle regole di business.

Passaggio 4: Conferma e Persistenza (confermaPrenotazione)

Il metodo confermaPrenotazione() di MatchUp ripropone alcuni controlli di sicurezza per garantire la consistenza dello stato:

- a) Esecuzione fuori sequenza (Nessuna prenotazione in corso): Invocazione del metodo senza che prenotaCampo() sia stato chiamato con successo in precedenza (stato prenotazioneCorrente == null). Esito atteso: false.
- b) Conferma con dati coerenti: Invocazione corretta al termine del flusso. Il metodo restituisce true, e la prenotazione viene salvata sia nella lista locale dell'oggetto Campo (listaPrenotazioniCampo), sia nel registro globale di MatchUp (catalogoPrenotazioni).